Git Cheat Sheet

GitVibes — Git for Vibe Coders · neovand.github.io/gitvibes

Setup & Config

```
git init
```
Initialize a new local repository

```
git init <directory>
```
Create a new repo in the specified directory

```
git clone <url>
```
Clone a remote repository to your machine

```
git clone --depth 1 <url>
```
Shallow clone with only the latest commit

```
git config --global user.name "<name>"
```
Set your name for all repositories

```
git config --global user.email "<email>"
```
Set your email for all repositories

```
git config --global core.editor "<editor>"
```
Set default text editor for Git

```
git config --list
```
List all current configuration settings

```
git config --global credential.helper <helper>
```
Store HTTPS credentials (osxkeychain, manager, libsecret)

```
ssh-keygen -t ed25519 -C "<email>"
```
Generate an SSH key pair for authentication

```
ssh -T git@github.com
```
Test your SSH connection to GitHub

```
gh auth login
```
Sign in to GitHub from the terminal (browser flow, no tokens to manage)

```
git config --global core.excludesFile
~/.gitignore_global
```
Global ignore file for OS and editor junk

```
git config core.hooksPath .githooks
```
Use a versioned, shareable hooks directory

Basic Workflow

```
git status
```
Show the working tree status

```
git add <file>
```
Stage a specific file for commit

```
git add .
```
Stage all changes in the current directory

```
git add -p
```
Interactively stage hunks of changes

```
git commit -m "<message>"
```
Commit staged changes with a message

```
git commit -am "<message>"
```
Stage and commit tracked files in one step

```
git commit --amend
```
Modify the most recent commit

```
git diff
```
Show unstaged changes in the working directory

```
git diff --staged
```
Show changes staged for the next commit

```
git rm --cached <file>
```
Stop tracking a file without deleting it (pair with .gitignore)

```
git commit --no-verify
```
Skip pre-commit and commit-msg hooks (use sparingly)

Branching

```
git branch
```
List all local branches

```
git branch -a
```
List all branches including remote

```
git branch <name>
```
Create a new branch

```
git branch -d <name>
```
Delete a fully merged branch

```
git branch -D <name>
```
Force delete a branch regardless of merge status

```
git switch <branch>
```
Switch to an existing branch

```
git switch -c <branch>
```
Create and switch to a new branch

```
git merge <branch>
```
Merge the specified branch into the current branch

Remote

```
git remote -v
```
List all remote connections with URLs

```
git remote add <name> <url>
```
Add a new remote repository

```
git remote remove <name>
```
Remove a remote connection

```
git remote rename <old> <new>
```
Rename an existing remote

```
git fetch <remote>
```
Download objects and refs from a remote

```
git fetch --prune
```
Fetch and remove stale remote-tracking branches

```
git pull <remote> <branch>
```
Fetch and merge changes from a remote branch

```
git pull --rebase
```
Fetch and replay your local commits on top (linear history)

```
git push <remote> <branch>
```
Push local commits to a remote branch

```
git push -u <remote> <branch>
```
Push and set the upstream tracking branch

```
git push --force-with-lease
```
Force push, but abort if someone else pushed first (safe force)

Stashing

```
git stash
```
Stash tracked changes (staged and unstaged; add -u for untracked files)

```
git stash push -m "<message>"
```
Stash changes with a descriptive message

```
git stash list
```
List all stashed changesets

```
git stash pop
```
Apply the latest stash and remove it from the list

```
git stash apply
```
Apply the latest stash but keep it in the list

```
git stash apply stash@{n}
```
Apply a specific stash by index

```
git stash drop stash@{n}
```
Remove a specific stash entry

```
git stash clear
```
Remove all stash entries

Undoing

```
git restore <file>
```
Discard changes in the working directory

```
git restore --staged <file>
```
Unstage a file while keeping changes

```
git reset <file>
```
Unstage a file (same as restore --staged)

```
git reset --soft HEAD~1
```
Undo last commit but keep changes staged

```
git reset --mixed HEAD~1
```
Undo last commit and unstage changes

```
git reset --hard HEAD~1
```
Undo last commit and discard all changes

```
git revert <commit>
```
Create a new commit that undoes a previous commit

```
git clean -fd
```
Remove untracked files and directories

History & Inspection

```
git log
```
Show the full commit history

```
git log --oneline
```
Show commit history in compact format

```
git log --oneline --graph --all
```
Visualize branch history as an ASCII graph

```
git log -p <file>
```
Show commit history with diffs for a file

```
git log --author="<name>"
```
Filter commits by author

```
git show <commit>
```
Display details and diff for a commit

```
git blame <file>
```
Show who last modified each line of a file

```
git diff <branch1>..<branch2>
```
Show differences between two branches

Rebase & Cherry-pick

```
git rebase <branch>
```

Reapply commits on top of another branch

```
git rebase -i HEAD~n
```

Interactively rebase the last n commits

```
git rebase --continue
```

Continue after resolving rebase conflicts

```
git rebase --abort
```

Cancel the rebase and return to original state

```
git cherry-pick <commit>
```

Apply a specific commit to the current branch

```
git cherry-pick <start>..<end>
```

Apply a range of commits (excludes <start> itself; use <start>^.. to include it)

```
git cherry-pick --continue
```

Resume a cherry-pick after resolving conflicts

```
git cherry-pick --abort
```

Cancel a conflicted cherry-pick and restore the branch

Tags

```
git tag
```

List all tags

```
git tag <name>
```

Create a lightweight tag at the current commit

```
git tag -a <name> -m "<message>"
```

Create an annotated tag with a message

```
git tag -a <name> <commit>
```

Tag a specific commit

```
git push <remote> <tag>
```

Push a specific tag to the remote

```
git push <remote> --tags
```

Push all tags to the remote

```
git tag -d <name>
```

Delete a local tag

Advanced

```
git bisect start
```

Begin a binary search to find a buggy commit

```
git bisect good <commit>
```

Mark a commit as good during bisect

```
git bisect bad <commit>
```

Mark a commit as bad during bisect

```
git bisect reset
```

End the bisect session and return to original HEAD

```
git reflog
```

Show a log of all reference updates (recovery tool)

```
git reset --hard HEAD@{1}
```

Move the branch back to where HEAD was one move ago

```
git submodule add <url> <path>
```

Add a repository as a submodule

```
git submodule update --init --recursive
```

Initialize and update all submodules

```
git worktree add <path> <branch>
```

Create a linked working tree for a branch

```
git worktree add -b <branch> <path>
```

Create a new branch in its own working tree

```
git worktree list
```

List all working trees for this repo

```
git worktree remove <path>
```

Remove a working tree (branch survives)

```
git worktree prune
```

Clean up records of deleted working trees